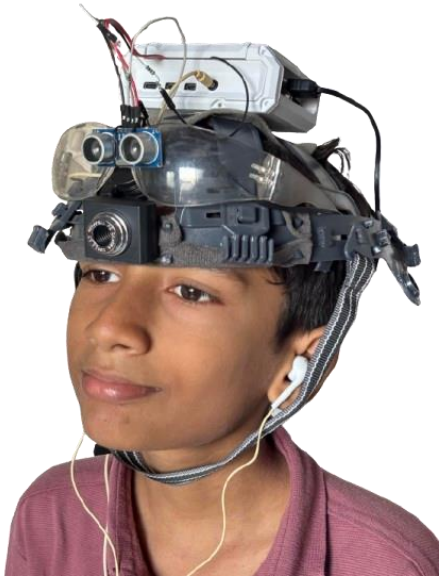
SMARTASSIST VISION

AI-POWERED MOBILITY AID

A Wearable Real-Time Object Detection and Obstacle Alert System for the Visually Impaired with Voice-Guided Navigation Through Earphones

Complete Build Guide

From Model Training & Testing on a Laptop to Full Deployment on a Raspberry Pi 4



The SmartAssist Vision prototype — Raspberry Pi 4, USB camera and HC-SR04 ultrasonic sensor mounted on a wearable helmet.

Built with: MobileNet-SSD • OpenCV • Raspberry Pi 4 • HC-SR04 • eSpeak / pyttsx3

SMARTASSIST VISION

AI-POWERED MOBILITY AID

A WEARABLE REAL-TIME OBJECT DETECTION AND OBSTACLE ALERT SYSTEM FOR THE VISUALLY IMPAIRED WITH VOICE-GUIDED NAVIGATION THROUGH EARPHONES

OBJECT DETECTION | OBSTACLE ALERT | DISTANCE MEASUREMENT | VOICE GUIDANCE | REAL-TIME PROCESSING

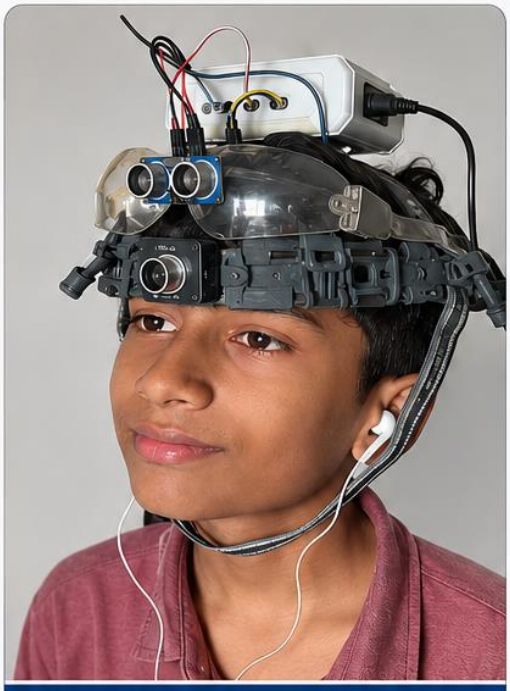
SYSTEM OVERVIEW

SmartAssist Vision is a wearable AI-powered assistive device designed for visually impaired individuals.

It detects objects and obstacles in real time using MobileNet-SSD, measures distance using an ultrasonic sensor, and provides voice-guided navigation through earphones.

KEY FEATURES

- Real-time object detection
- Voice-guided navigation
- Left / Center / Right object positioning
- Obstacle distance measurement
- Offline processing on Raspberry Pi 4
- Portable wearable design



SmartAssist Vision prototype — Raspberry Pi 4, USB camera and HC-SR04 ultrasonic sensor mounted on a wearable helmet.

SENSORS & HARDWARE USED

- Raspberry Pi 4
- USB Camera
- HC-SR04 Ultrasonic Sensor
- Earphones / Headphones
- Power Bank
- Wearable Helmet Mount

HOW IT WORKS

1. Camera captures surroundings
2. MobileNet-SSD detects objects
3. Ultrasonic sensor measures distance
4. Raspberry Pi processes data
5. Voice feedback delivered through earphones

SOFTWARE COMPONENTS

- Python
- OpenCV
- MobileNet-SSD
- eSpeak / pyttsx3
- Raspberry Pi OS

The system detects objects and obstacles in front, left, or right, calculates their distance, and guides the user through clear voice instructions for safe navigation.

Table of Contents

1. Introduction

Vision impairment affects an individual's ability to safely navigate everyday environments — detecting obstacles, recognizing nearby objects, and judging distances are tasks that sighted people take for granted. SmartAssist Vision is a low-cost, wearable assistive device built to address exactly this problem using affordable, off-the-shelf hardware and open-source software.

The system combines a real-time deep-learning object detector with an ultrasonic distance sensor and a text-to-speech voice engine, all running locally on a Raspberry Pi 4. The user wears the device on a helmet or headband; as they move, the system continuously scans the scene, identifies the most relevant nearby object, determines whether it is to the left, center, or right of the user, estimates how far away the nearest obstacle is, and announces this information through a Bluetooth or wired earpiece.

This document is a complete, beginner-friendly build guide. It walks through every stage of the project in the order it was actually developed: starting with model selection and software testing on a regular Windows laptop, and ending with full hardware integration and deployment on a Raspberry Pi 4. Every script referenced in this guide is included in full, along with explanations of how each piece fits into the overall system.

> *This guide assumes no prior experience with embedded systems or computer vision. Each step includes the exact commands and code used, so the project can be reproduced end-to-end.*

1.1 Why MobileNet-SSD?

The original concept for this project explored YOLOv10 for object detection. However, for a low-power single-board computer like the Raspberry Pi 4 (without a GPU or AI accelerator), YOLO models proved too heavy to run smoothly in real time. The project was therefore re-engineered around MobileNet-SSD — a lightweight Single Shot Detector built on the MobileNet backbone — which runs efficiently using OpenCV's built-in DNN module (cv2.dnn) on CPU alone.

MobileNet-SSD trades a wider object vocabulary for speed and efficiency. It can reliably detect 20 everyday object classes (person, chair, bottle, sofa, table, and more) at several frames per second on a Raspberry Pi 4 — fast enough for a real-time mobility aid.

2. Project Objectives

The SmartAssist Vision project was designed around the following core goals:

- Detect common objects and obstacles in front of the user in real time using a lightweight deep-learning model.
- Identify the relative position of the detected object — Left, Center, or Right — within the camera's field of view.
- Measure the distance to the nearest obstacle using an ultrasonic sensor, independent of the camera.
- Combine detection and distance data into short, natural spoken phrases delivered through headphones.
- Run the entire pipeline on a portable, battery-powered Raspberry Pi 4 — no internet connection or cloud processing required.
- Keep the total hardware cost low, using widely available components.

- Design the software so it can be developed and debugged on a laptop before being deployed to the Pi, minimizing on-device development time.

3. System Architecture

SmartAssist Vision is built around two parallel sensing pipelines that converge on the Raspberry Pi 4, which fuses their outputs into a single spoken message.

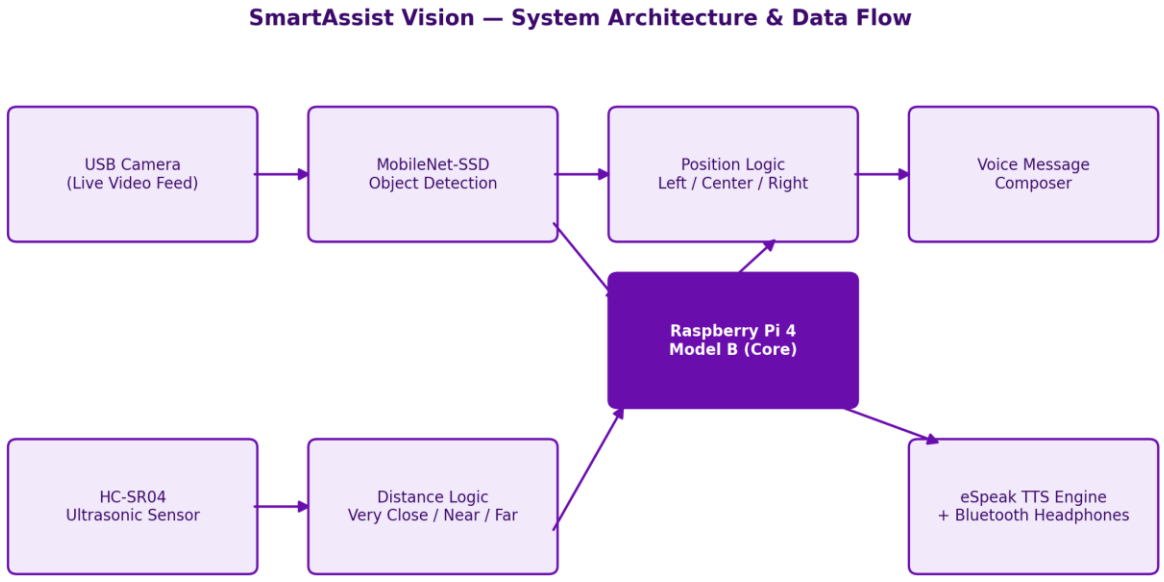


Figure 3.1 — High-level data flow of SmartAssist Vision.

3.1 Vision Pipeline

- A USB webcam continuously captures frames of the scene in front of the user.
- Each frame is resized to 300x300 pixels and passed through the MobileNet-SSD model via OpenCV's DNN module.
- The detection with the largest bounding box (the closest / most prominent object) is selected as the “primary” object for that frame.
- The horizontal center of that object's bounding box determines whether it lies in the Left, Center, or Right third of the frame.

3.2 Distance Pipeline

- An HC-SR04 ultrasonic sensor sends out a 40 kHz pulse and measures the time taken for the echo to return.
- This time is converted into a distance in centimeters and classified as Very Close (< 50 cm), Near (50-150 cm), or Far (> 150 cm).

3.3 Fusion & Voice Output

The Raspberry Pi combines the object label, its position, and the current distance category into a short phrase, for example “chair on left” or “caution table very close”. This phrase is converted to speech using eSpeak (on the Pi) or pyttsx3 (on the laptop) and played through Bluetooth or wired headphones. To avoid overwhelming the user, the system only speaks when the message changes, or repeats it after a short cooldown period if the same object remains in view.

4. Hardware Components / Bill of Materials

All components used in this build are inexpensive, widely available, and require no soldering beyond a simple voltage divider for the ultrasonic sensor.

Component	Purpose	Approx. Qty
Raspberry Pi 4 Model B (2GB/4GB)	Main processing unit running detection, sensor reading, and voice output	1
MicroSD Card (32GB, Class 10)	Stores Raspberry Pi OS and project files	1
USB Webcam (720p or 1080p)	Captures live video for object detection	1
HC-SR04 Ultrasonic Distance Sensor	Measures distance to the nearest obstacle	1
Resistors (1kΩ x2)	Voltage divider to safely connect the ECHO pin to the Pi's 3.3V GPIO	2
Jumper Wires (Male-to-Female / Male-to-Male)	Connecting the sensor to the Raspberry Pi GPIO header	Set
Bluetooth or Wired Headphones / Earpiece	Delivers spoken navigation feedback to the user	1
Power Bank (5V, 3A output, USB-C)	Portable power source for the Raspberry Pi	1
Helmet / Headband Mount	Wearable frame to hold the camera, sensor, and Pi	1
Laptop (Windows/Linux/Mac)	Used for initial development, testing, and writing the SD card image	1

Total estimated cost: under ₹6,000 / ~$75 USD, depending on region and whether components are already on hand.

5. Software & Tools Required

5.1 On the Laptop (Development & Testing)

- Windows 10/11 (or Linux/macOS)
- Visual Studio Code (or any code editor)
- Python 3.9 or later
- pip (Python package manager)
- Git (optional, for version control)

5.2 Python Libraries (Laptop)

These are installed inside a virtual environment, as shown in Section 7:

```
requirements.txt
comtypes==1.4.16
numpy==2.4.6
opencv-python==4.13.0.92
pypiwin32==223
pyttsx3==2.99
pywin32==312
```

5.3 On the Raspberry Pi

- Raspberry Pi OS (64-bit, Bullseye or newer) with Desktop
- Python 3 (pre-installed with Raspberry Pi OS)
- OpenCV (opencv-python)
- NumPy
- RPi.GPIO (pre-installed)
- eSpeak (text-to-speech engine, installed via apt)

5.4 MobileNet-SSD Model Files

Two files are required and used identically on both the laptop and the Raspberry Pi:

- MobileNetSSD_deploy.prototxt — defines the network architecture
- MobileNetSSD_deploy.caffemodel — contains the pre-trained weights (≈ 23 MB)

Both files are stored together in a folder named models/ inside the project directory. They are publicly available pre-trained Caffe model files commonly distributed alongside OpenCV's deep-learning samples.

6. Circuit Diagram & Wiring

The HC-SR04 ultrasonic sensor operates its ECHO pin at 5V, but the Raspberry Pi's GPIO pins are only rated for 3.3V. Connecting the ECHO pin directly would risk permanently damaging the Pi. A simple voltage divider made from two 1kΩ resistors steps the 5V echo signal down to a safe ~3.3V before it reaches GPIO 24.

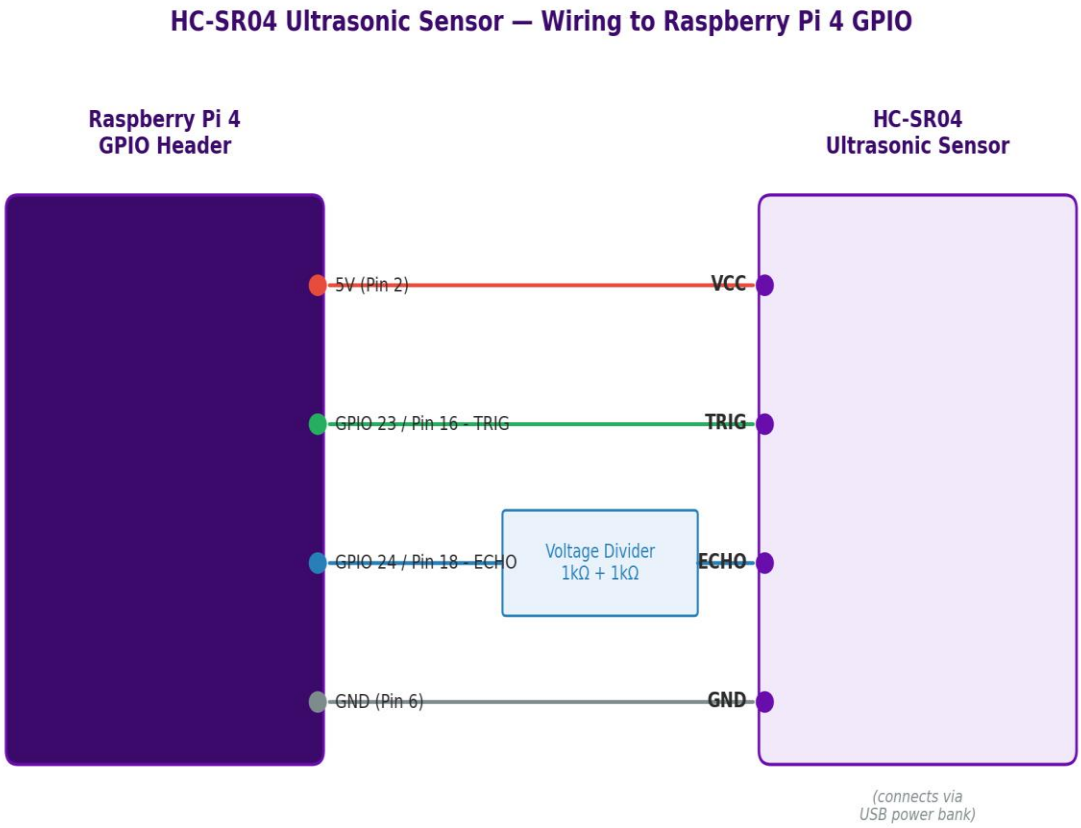


Figure 6.1 — HC-SR04 to Raspberry Pi 4 GPIO wiring, including the ECHO voltage divider.

HC-SR04 Pin	Connects To	Notes
VCC	Raspberry Pi 5V (Physical Pin 2)	Sensor power supply
TRIG	GPIO 23 (Physical Pin 16)	Sends the trigger pulse
ECHO	GPIO 24 (Physical Pin 18), via 1kΩ + 1kΩ divider	Returns the pulse width; must be stepped down from 5V to ~3.3V
GND	Raspberry Pi GND (Physical Pin 6)	Common ground

Build the voltage divider by connecting the ECHO pin to GPIO 24 through one 1kΩ resistor, and connecting the same GPIO 24 line to GND through a second 1kΩ resistor. This forms a simple two-resistor divider that halves the 5V echo signal to a safe level.

7. Phase 1 — Development & Testing on a Laptop

Before touching any hardware, the entire software pipeline was built and validated on a regular Windows laptop. This phase establishes the project structure, verifies the camera and model work correctly, and confirms the voice output — all without needing a Raspberry Pi.

7.1 Step 1: Create the Project Folder

Create a new project folder named SmartVision_Blind_Assistant with the following structure:

```
Project Structure
SmartVision_Blind_Assistant/
|
|-- models/
|     |-- MobileNetSSD_deploy.prototxt
|     |-- MobileNetSSD_deploy.caffemodel
|
|-- camera_test.py
|-- object_detection.py
|-- voice_output.py
|-- requirements.txt
|-- venv/
```

7.2 Step 2: Create a Virtual Environment

Open a terminal inside the project folder in VS Code and run:

```
Command Prompt (Windows)
cd SmartVision_Blind_Assistant
python -m venv venv
venv\Scripts\activate
```

Once activated, the terminal prompt should be prefixed with (venv), confirming the virtual environment is active.

7.3 Step 3: Install Required Libraries

```
Command Prompt (Windows)
pip install opencv-python
pip install numpy
pip install pyttsx3

pip freeze > requirements.txt
```

7.4 Step 4: Download the MobileNet-SSD Model

Download the two MobileNet-SSD model files (MobileNetSSD_deploy.prototxt and MobileNetSSD_deploy.caffemodel) and place them inside the models/ folder created in Step 1. These same two files will later be copied to the Raspberry Pi without any modification.

7.5 Step 5: Test the Camera

Before running any detection, confirm the webcam works correctly with OpenCV. Create camera_test.py:

```
camera_test.py
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        print("Camera not detected")
        break

    cv2.imshow("Pi Camera Test", frame)
```

```
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Run it with:

```
Command Prompt
python camera_test.py
```

A window should open showing a live feed from the webcam. Press 'q' to close it. A successful live feed confirms the camera and OpenCV installation are working correctly before moving on.

7.6 Step 6: Run MobileNet-SSD Object Detection

With the camera confirmed working, the next step integrates the MobileNet-SSD model using OpenCV's DNN module. Create object_detection.py:

```
object_detection.py
import cv2
import numpy as np

# MobileNetSSD classes
CLASSES = [
    "background", "aeroplane", "bicycle", "bird", "boat",
    "bottle", "bus", "car", "cat", "chair", "cow",
    "diningtable", "dog", "horse", "motorbike", "person",
    "pottedplant", "sheep", "sofa", "train", "tvmonitor"
]

# Load model
net = cv2.dnn.readNetFromCaffe(
    "models/MobileNetSSD_deploy.prototxt",
    "models/MobileNetSSD_deploy.caffemodel"
)

# USB Camera
cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()

    if not ret:
        print("Camera not detected")
        break

    (h, w) = frame.shape[:2]

    blob = cv2.dnn.blobFromImage(
        cv2.resize(frame, (300, 300)),
        scalefactor=0.007843,
        size=(300, 300),
        mean=127.5
    )

    net.setInput(blob)
    detections = net.forward()
```

```
    # Draw Left-Center-Right guide lines
    cv2.line(frame, (w // 3, 0), (w // 3, h), (255, 0, 0), 2)
    cv2.line(frame, (2 * w // 3, 0), (2 * w // 3, h), (255, 0, 0), 2)

    for i in range(detections.shape[2]):

        confidence = detections[0, 0, i, 2]

        if confidence > 0.6:

            idx = int(detections[0, 0, i, 1])

            box = detections[0, 0, i, 3:7] * np.array(
                [w, h, w, h]
            )

            (startX, startY, endX, endY) = box.astype("int")

            label = CLASSES[idx]

            center_x = (startX + endX) // 2

            if center_x < w // 3:
                position = "Left"
            elif center_x < 2 * w // 3:
                position = "Center"
            else:
                position = "Right"

            text = f"{label} - {position}"

            cv2.rectangle(
                frame,
                (startX, startY),
                (endX, endY),
                (0, 255, 0),
                2
            )

            cv2.putText(
                frame,
                text,
                (startX, startY - 10),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.6,
                (0, 255, 0),
                2
            )

    cv2.imshow("SmartVision Object Detection", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

This script performs the following, frame by frame:

1. Reads a frame from the webcam.
2. Resizes it to 300x300 pixels and converts it into a 'blob' — the input format expected by the MobileNet-SSD network.
3. Runs the blob through the network using net.forward(), producing a list of detections.
4. For every detection above a confidence threshold (0.6), draws a bounding box and label on the frame.
5. Determines whether each detected object's center falls in the left, center, or right third of the frame using two vertical guide lines drawn down the image.

Running this script opens a live window showing bounding boxes drawn around recognized objects such as people, chairs, bottles, sofas, and tables, each labelled with its name and screen position (Left / Center / Right).

7.7 Step 7: Position Detection Logic

The position-detection logic is the geometric core of the navigation system. The camera frame (of width w) is conceptually divided into three equal vertical zones:

- Left zone: x-coordinate from 0 to w/3
- Center zone: x-coordinate from w/3 to 2w/3
- Right zone: x-coordinate from 2w/3 to w

For each detected object, the bounding box's horizontal center (center_x = (startX + endX) / 2) is compared against these zone boundaries to decide whether the object is on the user's left, directly ahead, or on their right. This same logic is reused unchanged on the Raspberry Pi.

7.8 Step 8: Test Voice Output

With detection working, the final piece on the laptop is converting text into speech. Create voice_output.py:

voice_output.py

```
import pyttsx3

engine = pyttsx3.init()

engine.say("Person on the left")
engine.runAndWait()
```

Run python voice_output.py — the laptop's speakers should announce “Person on the left”. This confirms the pyttsx3 text-to-speech engine works correctly and is ready to be wired into the detection loop.

> *At this point, the laptop prototype can detect objects, classify their position, and speak — proving the full software concept before any Raspberry Pi hardware is involved.*

8. Phase 2 — Raspberry Pi Deployment

Once the software pipeline was validated on the laptop, the project was migrated to the Raspberry Pi 4 — the device that will actually be worn by the user. This phase covers OS setup, transferring the project, installing dependencies natively on the Pi, wiring the ultrasonic sensor, and switching the voice engine to one suited for embedded use.

8.1 Step 1: Prepare the Raspberry Pi OS

- Flash Raspberry Pi OS (64-bit, with Desktop) onto the microSD card using the Raspberry Pi Imager.
- Boot the Pi, complete the initial setup wizard, and connect it to Wi-Fi.
- Enable the camera and SSH from Raspberry Pi Configuration (sudo raspi-config), under Interface Options.
- Update the system packages:

```
Terminal (Raspberry Pi)
sudo apt update && sudo apt upgrade -y
```

8.2 Step 2: Create the Project Folder

```
Terminal (Raspberry Pi)
mkdir SmartVision_Blind_Assistant
cd SmartVision_Blind_Assistant
```

8.3 Step 3: Create a Virtual Environment

```
Terminal (Raspberry Pi)
python3 -m venv venv
source venv/bin/activate
```

8.4 Step 4: Install Required Libraries

```
Terminal (Raspberry Pi)
pip install opencv-python
pip install numpy
pip install RPi.GPIO
```

> *Installing opencv-python on a Raspberry Pi can take several minutes as some dependencies are compiled from source. If installation fails, try sudo apt install python3-opencv as an alternative, or install the additional system libraries OpenCV depends on (libatlas-base-dev, libjasper-dev, libqtgui4, etc.).*

8.5 Step 5: Copy the Project Files to the Pi

Transfer the entire project folder from the laptop to the Raspberry Pi — including the camera_test.py, object_detection.py, and the models/ folder containing MobileNetSSD_deploy.prototxt and MobileNetSSD_deploy.caffemodel. This can be done with a USB drive, scp over SSH, or by cloning a GitHub repository directly onto the Pi. The model files and detection code are used exactly as they were on the laptop — no changes are required for this step.

8.6 Step 6: Test the Camera on the Raspberry Pi

With the USB webcam connected to the Pi, re-run the same camera test script used in Phase 1 to confirm the camera initializes correctly on the new hardware:

```
camera_test.py
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
```

```
    if not ret:
        print("Camera not detected")
        break

    cv2.imshow("Pi Camera Test", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

A live preview window confirms the camera and OpenCV installation are working correctly on the Raspberry Pi.

8.7 Step 7: Run Object Detection on the Raspberry Pi

Next, the MobileNet-SSD detection script is run on the Pi using the same model files and detection logic from Phase 1. The Pi-adapted version below additionally tags each frame as Pi-based for clarity during testing:

object_detection_pi.py

```
import cv2
import numpy as np

# MobileNetSSD classes
CLASSES = [
    "background", "aeroplane", "bicycle", "bird", "boat",
    "bottle", "bus", "car", "cat", "chair", "cow",
    "diningtable", "dog", "horse", "motorbike", "person",
    "pottedplant", "sheep", "sofa", "train", "tvmonitor"
]

# Load model
net = cv2.dnn.readNetFromCaffe(
    "models/MobileNetSSD_deploy.prototxt",
    "models/MobileNetSSD_deploy.caffemodel"
)

# USB Camera
cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()

    if not ret:
        print("Camera not detected")
        break

    (h, w) = frame.shape[:2]

    blob = cv2.dnn.blobFromImage(
        cv2.resize(frame, (300, 300)),
        scalefactor=0.007843,
        size=(300, 300),
        mean=127.5
    )
```

```
    net.setInput(blob)
    detections = net.forward()

    # Draw Left-Center-Right guide lines
    cv2.line(frame, (w // 3, 0), (w // 3, h), (255, 0, 0), 2)
    cv2.line(frame, (2 * w // 3, 0), (2 * w // 3, h), (255, 0, 0), 2)

    for i in range(detections.shape[2]):

        confidence = detections[0, 0, i, 2]

        if confidence > 0.6:

            idx = int(detections[0, 0, i, 1])

            box = detections[0, 0, i, 3:7] * np.array(
                [w, h, w, h]
            )

            (startX, startY, endX, endY) = box.astype("int")

            label = CLASSES[idx]

            center_x = (startX + endX) // 2

            if center_x < w // 3:
                position = "Left"
            elif center_x < 2 * w // 3:
                position = "Center"
            else:
                position = "Right"

            text = f"{label} - {position}"

            cv2.rectangle(
                frame,
                (startX, startY),
                (endX, endY),
                (0, 255, 0),
                2
            )

            cv2.putText(
                frame,
                text,
                (startX, startY - 10),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.6,
                (0, 255, 0),
                2
            )

    cv2.imshow("SmartVision Object Detection", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Running this on the Pi produces the same real-time bounding boxes and Left / Center / Right labels seen on the laptop, confirming that MobileNet-SSD runs efficiently enough on the Pi's CPU for live use.

9. Ultrasonic Sensor Integration

The HC-SR04 sensor adds a second, independent layer of obstacle awareness — one that works even on objects the camera might miss (such as a wall or a low obstacle just below the camera's view), and gives a precise distance reading rather than just a bounding-box size estimate.

9.1 How the HC-SR04 Works

The sensor is triggered with a short 10-microsecond HIGH pulse on its TRIG pin. It then emits a 40 kHz ultrasonic burst and raises its ECHO pin HIGH for exactly as long as it takes for the echo to return. By measuring how long ECHO stays HIGH and multiplying by the speed of sound, the distance to the nearest object directly ahead of the sensor can be calculated.

9.2 Distance Test Script

After wiring the sensor as shown in Section 6, the following script reads continuous distance measurements and classifies them:

distance_test.py

```
import RPi.GPIO as GPIO
import time

TRIG = 23
ECHO = 24

GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)

GPIO.output(TRIG, False)

print("Waiting for sensor...")
time.sleep(2)

try:
    while True:

        # Send trigger pulse
        GPIO.output(TRIG, True)
        time.sleep(0.00001)
        GPIO.output(TRIG, False)

        pulse_start = time.time()
        pulse_end = time.time()

        # Wait for echo to go HIGH
        timeout = time.time()
        while GPIO.input(ECHO) == 0:
            pulse_start = time.time()

            if time.time() - timeout > 0.05:
```

```
                break

        # Wait for echo to go LOW
        timeout = time.time()
        while GPIO.input(ECHO) == 1:
            pulse_end = time.time()

            if time.time() - timeout > 0.05:
                break

        pulse_duration = pulse_end - pulse_start

        distance = pulse_duration * 17150
        distance = round(distance, 2)

        # Ignore invalid readings
        if distance < 2 or distance > 400:
            continue

        # Distance classification
        if distance < 50:
            status = "Very Close"
        elif distance < 150:
            status = "Near"
        else:
            status = "Far"

        print(f"Distance: {distance} cm | Status: {status}")

        time.sleep(0.5)

except KeyboardInterrupt:
    print("\nProgram stopped")
    GPIO.cleanup()
```

9.3 Distance Classification

Measured Distance	Status	Typical Voice Cue
Less than 50 cm	Very Close	“caution [object] very close”
50 cm to 150 cm	Near	“[object] on [left/center/right]”
More than 150 cm	Far	“[object] ahead”

Readings below 2 cm or above 400 cm are discarded as out-of-range / invalid sensor noise, and the loop simply tries again on the next cycle.

10. Voice Feedback Integration

On the laptop, pyttsx3 (used in Phase 1, Step 8) worked well because it relies on the Windows SAPI5 speech engine. On the Raspberry Pi, however, pyttsx3 produced compatibility and audio-driver issues. The project switched to eSpeak — a lightweight, offline, open-source text-to-speech synthesizer that runs reliably on Linux-based systems.

10.1 Installing eSpeak

```
Terminal (Raspberry Pi)
```

```
sudo apt install espeak
```

10.2 Voice Test Script

The following script confirms eSpeak is working by speaking a series of sample navigation phrases — the same kind of messages the final system will generate automatically:

voice_test.py

```
import os
import time

messages = [
    "Person on the left near",
    "Chair in center very close",
    "Bottle on the right far"
]

for msg in messages:
    print(msg)
    os.system(f'espeak "{msg}"')
    time.sleep(2)
```

10.3 Routing Audio to Headphones

Bluetooth headphones were paired with the Raspberry Pi using the Bluetooth manager (bluetoothctl), and set as the default audio output device. eSpeak's audio then plays automatically through the paired headphones, giving the user private, hands-free navigation feedback.

Example eSpeak command

```
espeak -s 120 "Person ahead"
```

> *The -s 120 flag sets the speaking rate (words per minute). A slightly slower rate improves clarity when listening through headphones while walking.*

11. Final System Integration

With the camera pipeline, ultrasonic sensor, and voice engine each working independently, the final step combines them into a single program: smartvision_final.py. This is the script that runs continuously on the Raspberry Pi during normal use.

11.1 Design Goals of the Final Script

- Run the camera and detection loop continuously without blocking on sensor or speech delays.
- Re-check the ultrasonic distance roughly every 0.5 seconds, rather than every single frame, since it does not need to update as fast as the camera.
- Run object detection on every other frame (frame_count % 2 == 0) to keep the overall loop responsive on the Pi's CPU.
- Choose the spoken phrase based on both the detected object's position and the current distance category.
- Avoid repeating the same announcement too frequently, using a short 'candidate message' delay plus a cooldown timer.

- Speak using eSpeak in a non-blocking way (os.system(...) with a trailing &) so the main loop never freezes while audio plays.

11.2 Key Logic Walkthrough

The script defines a get_distance() function that performs the same TRIG/ECHO pulse measurement seen in Section 9, returning None if the reading times out or is out of the valid 2-400 cm range.

Inside the main loop, the current distance status (very close / near / far / unknown) is used to choose how the detected object is described:

- If status is “far” → the message is “[object] ahead”.
- If status is “near” → the message is “[object] on [left/center/right]”.
- Otherwise (very close) → the message becomes “caution [object] very close”, prioritizing safety over positional detail.

A small dictionary, DISPLAY_NAMES, renames a few MobileNet-SSD class labels into more natural spoken words — for example, “diningtable” becomes “table”, “tvmonitor” becomes “monitor”, and “pottedplant” becomes “plant”.

11.3 Speech Timing Logic

To prevent the device from chattering constantly, the script uses two timers:

- SPEAK_DELAY (0.7 seconds): a new message must remain stable for this long before it is spoken, filtering out flickering detections.
- COOLDOWN (2 seconds): once a message is spoken, the same message will not be repeated again until this cooldown has elapsed, even if the object remains in view.

11.4 Complete Final Script

The full, final integrated script is reproduced below. It brings together the ultrasonic sensor, MobileNet-SSD detection, position logic, distance-aware phrasing, and eSpeak voice output into the single program that runs on the wearable device.

smartvision_final.py

```
import cv2
import numpy as np
import RPi.GPIO as GPIO
import time
import os

# ====================================
# HC-SR04 SETUP
# ====================================

TRIG = 23
ECHO = 24

GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)

GPIO.output(TRIG, False)
time.sleep(2)
```

```
# ====================================
# MobileNetSSD Classes
# ====================================

CLASSES = [
    "background", "aeroplane", "bicycle", "bird", "boat",
    "bottle", "bus", "car", "cat", "chair", "cow",
    "diningtable", "dog", "horse", "motorbike", "person",
    "pottedplant", "sheep", "sofa", "train", "tvmonitor"
]

DISPLAY_NAMES = {
    "diningtable": "table",
    "tvmonitor": "monitor",
    "motorbike": "bike",
    "pottedplant": "plant"
}

# ====================================
# LOAD MODEL
# ====================================

net = cv2.dnn.readNetFromCaffe(
    "models/MobileNetSSD_deploy.prototxt",
    "models/MobileNetSSD_deploy.caffemodel"
)

# ====================================
# DISTANCE FUNCTION
# ====================================

def get_distance():

    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)

    pulse_start = time.time()
    pulse_end = time.time()

    timeout = time.time()

    while GPIO.input(ECHO) == 0:
        pulse_start = time.time()

        if time.time() - timeout > 0.05:
            return None

    timeout = time.time()

    while GPIO.input(ECHO) == 1:
        pulse_end = time.time()

        if time.time() - timeout > 0.05:
            return None

    pulse_duration = pulse_end - pulse_start

    distance = pulse_duration * 17150
    distance = round(distance, 2)
```

```
    if distance < 2 or distance > 400:
        return None

    return distance

# ====================================
# CAMERA
# ====================================

cap = cv2.VideoCapture(0)

cap.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 240)

# ====================================
# SPEECH SETTINGS
# ====================================

last_message = ""
last_spoken_time = 0

candidate_message = ""
candidate_start_time = 0

SPEAK_DELAY = 0.7
COOLDOWN = 2

# ====================================
# DISTANCE CACHE
# ====================================

distance = None
status = "unknown"
last_distance_time = 0

# ====================================
# DETECTION CACHE
# ====================================

best_message = ""
best_box = None

frame_count = 0

try:

    while True:

        ret, frame = cap.read()

        if not ret:
            print("Camera not detected")
            break

        (h, w) = frame.shape[:2]

        current_time = time.time()
```

```
        # ============================
        # DISTANCE UPDATE
        # ============================

        if current_time - last_distance_time > 0.5:

            distance = get_distance()

            if distance is None:
                status = "unknown"

            elif distance < 50:
                status = "very close"

            elif distance < 150:
                status = "near"

            else:
                status = "far"

            last_distance_time = current_time

        # ============================
        # OBJECT DETECTION
        # ============================

        frame_count += 1

        if frame_count % 2 == 0:

            blob = cv2.dnn.blobFromImage(
                cv2.resize(frame, (300, 300)),
                0.007843,
                (300, 300),
                127.5
            )

            net.setInput(blob)
            detections = net.forward()

            largest_area = 0
            current_message = ""
            current_box = None

            for i in range(detections.shape[2]):

                confidence = detections[0, 0, i, 2]

                if confidence < 0.60:
                    continue

                idx = int(detections[0, 0, i, 1])

                label = CLASSES[idx]
                label = DISPLAY_NAMES.get(label, label)

                box = detections[0, 0, i, 3:7] * np.array(
                    [w, h, w, h]
                )
```

```
            (startX, startY, endX, endY) = box.astype("int")

            area = (endX - startX) * (endY - startY)

            if area > largest_area:

                largest_area = area

                center_x = (startX + endX) // 2

                if center_x < w // 3:
                    position = "left"
                elif center_x < 2 * w // 3:
                    position = "center"
                else:
                    position = "right"

                if status == "far":
                    current_message = f"{label} ahead"

                elif status == "near":
                    current_message = f"{label} on {position}"

                else:
                    current_message = f"caution {label} very close"

                current_box = (
                    startX,
                    startY,
                    endX,
                    endY
                )

        if current_message:
            best_message = current_message
            best_box = current_box

    # ============================
    # SPEECH LOGIC
    # ============================

    current_time = time.time()

    if best_message:

        if best_message != candidate_message:

            candidate_message = best_message
            candidate_start_time = current_time

        elif (
            current_time - candidate_start_time >= SPEAK_DELAY
            and current_time - last_spoken_time >= COOLDOWN
            and best_message != last_message
        ):

            print("Speaking:", best_message)
```

```
                os.system(
                    f'espeak -s 120 "{best_message}" >/dev/null 2>&1 &'
                )

                last_message = best_message
                last_spoken_time = current_time

        # ============================
        # DISPLAY
        # ============================

        cv2.line(frame, (w // 3, 0), (w // 3, h), (255, 0, 0), 1)
        cv2.line(frame, (2 * w // 3, 0), (2 * w // 3, h), (255, 0, 0), 1)

        if distance is not None:

            cv2.putText(
                frame,
                f"{distance:.1f} cm | {status}",
                (10, 20),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.5,
                (0, 0, 255),
                2
            )

        if best_box is not None:

            startX, startY, endX, endY = best_box

            cv2.rectangle(
                frame,
                (startX, startY),
                (endX, endY),
                (0, 255, 0),
                2
            )

            cv2.putText(
                frame,
                best_message,
                (startX, max(20, startY - 10)),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.45,
                (0, 255, 0),
                2
            )

        cv2.imshow("Smart Vision Assistant", frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

finally:

    cap.release()
    cv2.destroyAllWindows()
    GPIO.cleanup()
```

12. Running the Final System

To run the complete SmartAssist Vision system on the Raspberry Pi:

6. Power on the Raspberry Pi and wait for it to fully boot.
7. Connect / pair the Bluetooth headphones (or plug in wired headphones) before starting the program.
8. Activate the virtual environment: source venv/bin/activate
9. Navigate to the project folder: cd SmartVision_Blind_Assistant
10. Run the final script: python smartvision_final.py
11. Wear the device and move around — the system will begin announcing detected objects, their position, and obstacle distance through the headphones.
12. Press 'q' on the preview window (if a display is attached) or Ctrl+C in the terminal to stop the program safely. The script's finally block ensures the camera and GPIO pins are released cleanly on exit.

> *For everyday wearable use without a monitor, the cv2.imshow() preview window can be disabled and the script can be set to launch automatically on boot using a systemd service or a crontab @reboot entry.*

13. Testing & Results

The system was tested in an indoor environment with common objects placed at varying distances and angles from the wearer. The table below summarizes the key results observed during testing.

Test	Expected Behaviour	Result
Camera initialization (laptop & Pi)	Live video feed opens without errors	Pass
MobileNet-SSD detection (laptop & Pi)	Bounding boxes drawn around known objects with confidence > 0.6	Pass
Left / Center / Right classification	Position label matches the object's actual location in frame	Pass
HC-SR04 distance reading	Readings within 2-400 cm classified as Very Close / Near / Far	Pass
Voice output (pyttsx3, laptop)	Spoken phrase matches detected text	Pass
Voice output (eSpeak, Pi)	Spoken phrase plays through Bluetooth headphones	Pass
Combined real-time loop on Pi 4	Detection + distance + speech run together without freezing	Pass
Repeat-message suppression	Same message is not repeated within the cooldown window	Pass

Overall, the system successfully detected objects in real time, determined their position relative to the user, measured obstacle distance independently of the camera, and generated clear, timely voice guidance through headphones — running entirely on a Raspberry Pi 4 powered by a portable battery bank.

14. Limitations

- MobileNet-SSD recognizes only 20 object classes and cannot detect items such as mobile phones, stairs, or curbs — common everyday hazards.
- A single, forward-facing ultrasonic sensor cannot measure the distance to every object the camera sees; it only reports the nearest obstacle directly ahead.
- Detection accuracy decreases in low-light conditions, since the system relies on a standard RGB camera.
- The current prototype processes one detection per alternating frame to maintain real-time performance, which can slightly delay recognition of fast-moving objects.
- Voice announcements are limited to short, templated phrases rather than full natural-language descriptions of the scene.

15. Future Scope

- Upgrade to a more capable model (e.g. YOLOv8/YOLOv10 with hardware acceleration such as a Coral USB Accelerator or Hailo module) to detect a wider range of objects without sacrificing speed.
- Add multiple ultrasonic or ToF sensors arranged around the headset for 360-degree obstacle awareness.
- Implement object tracking across frames to provide smoother, more continuous guidance instead of per-frame announcements.
- Integrate GPS and a digital compass for outdoor turn-by-turn navigation assistance.
- Add a small vibration motor (haptic feedback) as a silent alternative or complement to voice alerts, useful in noisy environments.
- Develop a companion mobile app for configuration, battery monitoring, and emergency location sharing.
- Optimize power consumption to extend battery life for full-day wearable use.

16. Conclusion

SmartAssist Vision demonstrates that a genuinely useful, real-time mobility aid for visually impaired users can be built from low-cost, widely available hardware and entirely open-source software. By developing and validating each component — camera input, MobileNet-SSD object detection, position classification, ultrasonic distance sensing, and text-to-speech output — independently on a laptop before integrating them on a Raspberry Pi 4, the project minimized on-device debugging and produced a robust, working wearable prototype.

The final system runs continuously on battery power, requires no internet connectivity, and provides the user with concise, spoken information about nearby objects, their position, and how close they are. While there is plenty of room for future enhancement — broader object recognition, multi-sensor obstacle mapping, and haptic feedback among them — the current prototype already fulfils its core mission: helping a visually impaired person move through an indoor space with greater confidence and independence.

Appendix A: requirements.txt

Python dependencies used during laptop development (Phase 1):

requirements.txt

```
comtypes==1.4.16
numpy==2.4.6
opencv-python==4.13.0.92
pypiwin32==223
pyttsx3==2.99
pywin32==312
```

Appendix B: Project File Reference

File	Description
camera_test.py	Verifies the USB webcam feed using OpenCV.
object_detection.py / object_detection_pi.py	MobileNet-SSD detection with Left/Center/Right position classification.
voice_output.py	Tests text-to-speech output using pyttsx3 (laptop).
voice_test.py	Tests text-to-speech output using eSpeak (Raspberry Pi).
distance_test.py	Reads and classifies distance from the HC-SR04 ultrasonic sensor.
smartvision_final.py	The complete, integrated wearable application — combines detection, distance sensing, and voice output.
MobileNetSSD_deploy.prototxt	MobileNet-SSD network architecture definition.
MobileNetSSD_deploy.caffemodel	Pre-trained MobileNet-SSD model weights (~23 MB).